

Patrones implementados – Voz del Ciudadano

En este documento se busca explicar los patrones usados, que hacen y su importancia.

Importante especificar que cuando se menciona al Congreso o a la RENIEC, se usaron mocks que simulan estos procesos que necesitarían un API real ya en producción.

Sergio Sebastian Ninasivinchá Ore – 20232692C

1. Facade — InitiativeFacade

Archivo: pc3-backend/app/patterns/facade.py

Que hace

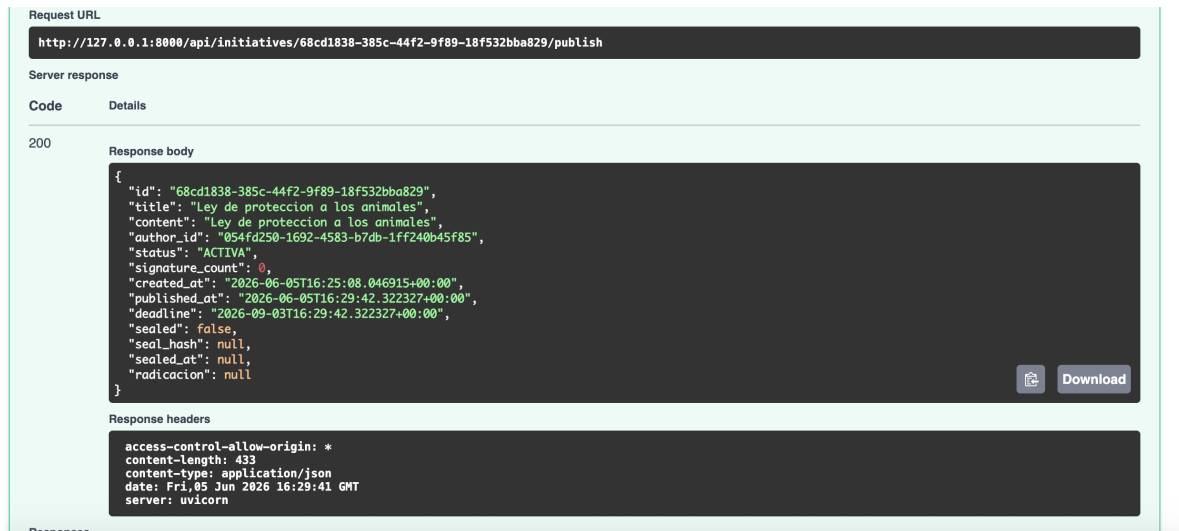
El patrón Facade provee una interfaz simplificada sobre un conjunto de subsistemas complejos. En lugar de que el cliente deba conocer y coordinar cada componente por separado, interactúa con un único objeto que delega internamente.

Funcionamiento en el sistema

InitiativeFacade centraliza todas las operaciones sobre una iniciativa legislativa. Cuando un colectivo publica una iniciativa, no es necesario que la capa de rutas conozca el orden exacto de validación, la lógica del temporizador, el registro en auditoría o el proceso de envío al Congreso. La Facade orquesta cada uno de esos pasos en el orden correcto a través de sus métodos `publish`, `sign` y `seal_and_dispatch`. Internamente, coordina con el Adapter para el envío, con el Proxy para el sellado y con el registro de auditoría para la trazabilidad.

Por qué es importante

Sin la Facade, cada endpoint de la API tendría que conocer el orden de operaciones interno del sistema y manejar manualmente la consistencia entre subsistemas. Cualquier cambio en la lógica de negocio obligaría a modificar múltiples rutas. La Facade encapsula esa complejidad en un único punto, facilitando el mantenimiento y reduciendo el acoplamiento entre la capa HTTP y la lógica de dominio.



2. Adapter — ReniecAdapter

Archivo: pc3-backend/app/patterns/adapter.py

Que hace

El patrón Adapter convierte la interfaz de un componente externo en la interfaz que el sistema espera, permitiendo que dos componentes incompatibles trabajen juntos sin modificarse mutuamente.

Funcionamiento en el sistema

El servicio externo RENIEC expone un método `consultar_identidad(numero_documento)` que devuelve un diccionario con campos propios (`valido`, `estado`, `num_doc`). El sistema interno, en cambio, espera una interfaz `IdentityVerifier` con un método `verify(identifier) -> bool`. `ReniecAdapter` actúa como puente: recibe la llamada estandarizada `verify`, la traduce al formato que entiende `ReniecServiceStub`, interpreta la respuesta y devuelve un booleano limpio. Lo mismo ocurre con `CongressApiAdapter`, que adapta el envío del expediente a la API del Congreso. En ambos casos se trabaja contra un stub que simula el servicio real.

Por que es importante

Los servicios externos cambian con frecuencia: pueden modificar su API, sus campos o su protocolo. Si el sistema acoplara directamente la lógica de negocio al formato del servicio externo, cada cambio en RENIEC o en la API del Congreso obligaría a modificar múltiples partes del código. Con el Adapter, solo se actualiza la clase adaptadora y el resto del sistema permanece intacto.

http://127.0.0.1:8000/api/auth/register

Server response

Code	Details
200	Response body <pre>{ "id": "76a8763a-8c19-471c-adb0-68c4eba652cb", "name": "Juan", "email": "juan@example.com", "dni": "46722747", "verified": true }</pre> Response headers <pre>access-control-allow-origin: * content-length: 119 content-type: application/json date: Fri, 05 Jun 2026 16:13:17 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

3. Decorator — DuplicateCheckDecorator y MetadataEnrichmentDecorator

Archivo: pc3-backend/app/patterns/decorator.py

Que hace

El patrón Decorator añade comportamiento adicional a un objeto de forma dinámica, envolviéndolo en otro objeto que comparte la misma interfaz. Permite extender funcionalidades sin modificar la clase original ni usar herencia.

Funcionamiento en el sistema

El proceso de firma parte de BaseSignatureService, que simplemente registra la firma.

Sobre este servicio base se apilan dos decoradores en cadena. Primero, DuplicateCheckDecorator intercepta la llamada y verifica si el ciudadano ya firmó esa iniciativa; si es así, lanza un error antes de continuar. Si pasa esa validación, MetadataEnrichmentDecorator enriquece la firma con un hash SHA-256 del contenido del documento en ese instante y un timestamp UTC, garantizando integridad y trazabilidad. Solo entonces se delega al servicio base para persistir la firma. Cada decorador es independiente y puede añadirse o retirarse sin afectar a los demás.

Por que es importante

Las validaciones y enriquecimientos de una firma son responsabilidades separadas que no deberían mezclarse en una sola clase. El Decorator permite combinarlas de forma modular: si en el futuro se requiere validar geolocalización o añadir firma biométrica, basta con crear un nuevo decorador y envolverlo en la cadena, sin tocar el código existente.

200	Response body <pre>{ "signature": { "id": "fc9ba496-0f8b-4b1e-a31c-84b2e49279d9", "user_id": "054fd250-1692-4583-b7db-1ff240b45f85", "initiative_id": "68cd1838-385c-44f2-9f89-18f532bba829", "status": "VALIDA", "doc_hash": "f29bf07e51530446ce25f17fd7a3c07121da050766825da91b846e81e521c8de", "timestamp": "2026-06-05T16:33:19.037510+00:00" }, "signature_count": 1, "initiative_status": "ACTIVA" }</pre> Response headers <pre>access-control-allow-origin: * content-length: 355 content-type: application/json date: Fri, 05 Jun 2026 16:33:18 GMT server: uvicorn</pre>	
Responses		
Code	Description	Links
200	Successful Response	No links

4. Proxy — CryptographicSealProxy

Archivo: pc3-backend/app/patterns/proxy.py

Que hace

El patrón Proxy interpone un objeto intermediario que controla el acceso a otro objeto real. El cliente cree que interactúa con el objeto real, pero en realidad pasa primero por el Proxy, que puede añadir lógica de control, seguridad o caché.

Funcionamiento en el sistema

InitiativeExpedient es el objeto real que representa el expediente de una iniciativa y expone operaciones de lectura y escritura. CryptographicSealProxy envuelve ese expediente y, antes de permitir cualquier escritura, verifica si el expediente ya fue sellado criptográficamente. Si lo está, rechaza la operación con un error y registra el intento en el log de auditoría. Si no lo está, genera el hash SHA-512 del contenido completo, congela el expediente como solo lectura y delega al objeto real para persistir el sello. De esta forma, la inmutabilidad del expediente sellado es una garantía estructural y no depende de la disciplina del código que lo llama.

Por que es importante

Un expediente legislativo sellado no puede ser alterado bajo ninguna circunstancia. Sin el Proxy, cualquier parte del sistema podría modificar accidentalmente un expediente ya enviado al Congreso. El Proxy centraliza el control de acceso en un único punto, haciendo imposible que esa regla sea ignorada u omitida por error.

ENVIADA Radicacion: RAD-68CD1838

Ley de proteccion a los animales

da6fbd70e73d4241f1fce0fbd626d8d2d2967a6457cff443017c9f68ff05239982327f6f8ffccc59...

Sellado: 5/6/2026, 11:38:18

Firmar esta iniciativa

Escribe un comentario...

Comentar

Response body

```

    "detail": "2026-06-05T16:25:08.046972+00:00"
  },
  {
    "action": "PUBLISH",
    "actor": "054fd250-1692-4583-b7bd-1ff240b45f85",
    "entity_id": "68cd1838-385c-44f2-9f89-18f532bba829",
    "detail": "",
    "timestamp": "2026-06-05T16:29:42.322369+00:00"
  },
  {
    "action": "SIGN",
    "actor": "054fd250-1692-4583-b7bd-1ff240b45f85",
    "entity_id": "68cd1838-385c-44f2-9f89-18f532bba829",
    "detail": "",
    "timestamp": "2026-06-05T16:33:19.037529+00:00"
  },
  {
    "action": "SEAL_AND_SEND",
    "actor": "system",
    "entity_id": "68cd1838-385c-44f2-9f89-18f532bba829",
    "detail": "hash=da6fbfd70e73d4241...",
    "timestamp": "2026-06-05T16:38:18.536289+00:00"
  }
]

```

Response headers

```
content-length: 1815
content-type: application/json
date: Fri,05 Jun 2026 16:51:53 GMT
server: uvicorn
```

5. Composite — CommentBranch y CommentLeaf

Archivo: pc3-backend/app/patterns/composite.py

Que hace

El patrón Composite organiza objetos en estructuras de árbol para representar jerarquías parte-todo. Permite tratar de forma uniforme tanto a los objetos individuales (hojas) como a los contenedores (ramas), ya que ambos comparten la misma interfaz.

Funcionamiento en el sistema

Los comentarios de una iniciativa forman un árbol de hasta 3 niveles de profundidad. CommentBranch representa un comentario que puede recibir respuestas (niveles 1 y 2); CommentLeaf representa un comentario terminal que no admite hijos (nivel 3). Ambos implementan la misma interfaz con el método to_dict(), por lo que el sistema puede serializar todo el árbol de forma recursiva sin distinguir si está recorriendo una rama o una hoja. La función build_comment decide qué tipo de nodo crear según la profundidad y lanza un error si se supera el límite.

Por que es importante

Sin el Composite, el sistema necesitaría lógica condicional dispersa en múltiples lugares para tratar diferente a los comentarios con hijos y sin hijos. El patrón permite que la serialización, la consulta y la construcción del árbol operen de forma uniforme sobre cualquier nodo, independientemente de su posición en la jerarquía. Además, el límite de profundidad queda encapsulado en la lógica de construcción, no en las rutas HTTP.

http://127.0.0.1:8000/api/initiatives/08cd1838-385c-44f2-9f89-18f532bba829/comments

Server response

Code	Details
200	Response body <pre>{ "initiative_id": "68cd1838-385c-44f2-9f89-18f532bba829", "comments": [{ "id": "43839442-928c-40fe-9e4c-082206c8a977", "text": "Apoyo esta iniciativa firmemente.", "author_id": "054fd250-1692-4583-b7db-1ff240b45f85", "parent_id": null, "level": 1, "created_at": "2026-06-05T16:36:43.831978+00:00", "children": [{ "id": "f4d68b53-1b75-4858-9093-5b5948c7eb59", "text": "Concuerdo", "author_id": "054fd250-1692-4583-b7db-1ff240b45f85", "parent_id": "43839442-928c-40fe-9e4c-082206c8a977", "level": 2, "created_at": "2026-06-05T16:37:21.679935+00:00", "children": [{ "id": "bdd20afc-5bd3-4902-a9aa-138cb9b9d09d", "text": "Juntos podemos", "author_id": "054fd250-1692-4583-b7db-1ff240b45f85", "parent_id": "f4d68b53-1b75-4858-9093-5b5948c7eb59", "level": 3, "created_at": "2026-06-05T16:37:21.679935+00:00", "children": [] }] }] }] }</pre> Response headers

Download